第7讲 故障恢复：日志、WAL 与 ARIES

《数据库系统原理》2026 秋 · 课件

故障的种类

- **事务故障**：事务内部出错（如违反约束、除零）或被死锁检测选为牺牲者，事务无法继续，需要回滚。
- **系统故障**：掉电、操作系统崩溃等，内存（缓冲区）中的内容全部丢失，但磁盘数据完好。
- **介质故障**：磁盘损坏，持久存储上的数据丢失，需要从备份和归档日志恢复。
- 恢复子系统的目标：故障后把数据库恢复到一个**一致的状态**，保证事务的**原子性**和**持久性**。

日志记录与日志文件

- 日志按时间顺序**追加**写入，每条日志有一个单调递增的日志序列号 **LSN**。
- 更新日志记录：`<T，X，旧值，新值>`，旧值用于撤销，新值用于重做。
- 事务开始、提交、回滚分别记 `<T start>`、`<T commit>`、`<T abort>`。
- 日志先写在内存的日志缓冲区，再按规则刷到稳定存储（磁盘）上。

先写日志原则（WAL）

- **Write-Ahead Logging**：一个数据页被写回磁盘之前，描述该页修改的日志记录必须**先**写到稳定存储。
- 事务提交之前，它的所有日志记录（包括 commit 记录）必须先刷到磁盘，提交才算完成。
- 第一条保证崩溃后总能找到旧值来 **undo**；第二条保证已提交事务总能找到新值来 **redo**。
- 有了 WAL，缓冲区管理可以采用 **steal**（未提交的脏页也可写回）+ **no-force**（提交时不必强制写数据页）的策略，性能最好。

Undo 与 Redo

- steal 策略下，未提交事务的修改可能已经落盘 → 恢复时要 **undo**。
- no-force 策略下，已提交事务的修改可能还没落盘 → 恢复时要 **redo**。
- 基本恢复过程：正向扫描日志，对有 commit 记录的事务重做；反向扫描日志，对没有 commit 也没有 abort 的事务撤销。
- undo 和 redo 都必须是**幂等**的：恢复过程中再次崩溃，重新执行一遍结果不变。

检查点（Checkpoint）

- 不做检查点的话，恢复时要从日志开头扫描，太慢。
- 检查点把当前活跃事务表、脏页信息写入日志，恢复时只需从**最近的检查点**附近开始扫描。
- **模糊检查点**（fuzzy checkpoint）：做检查点时不暂停正在运行的事务，也不强制把所有脏页刷盘，只记录状态。

ARIES 恢复算法的三个阶段

- **分析（Analysis）**：从最近的检查点正向扫描日志，重建**脏页表**和**活跃事务表**，确定 redo 的起点（脏页表中最小的 recLSN）。
- **重做（Redo）**：从该起点开始"**重演历史**"，把所有更新（包括未提交事务的更新）都重做一遍，使数据库回到崩溃前一刻的状态。
- **撤销（Undo）**：反向回滚崩溃时仍未提交的事务，每撤销一个操作写一条**补偿日志记录** CLR。

LSN、pageLSN 与补偿日志记录（CLR）

- 每个数据页上记录 **pageLSN**：最后一次修改该页的日志 LSN。redo 时若日志 LSN ⩽ pageLSN，说明修改已在页上，跳过。
- **CLR** 记录"撤销了哪条日志"，并带 **UndoNextLSN** 指向下一条要撤销的日志。
- CLR 只 redo、不 undo：undo 过程中再次崩溃时，已经撤销的部分不会被重复撤销，恢复一定能结束。

本讲小结

- 故障分三类；恢复保证原子性和持久性。
- WAL 是一切的前提：先写日志，后写数据页；提交前日志先落盘。
- ARIES = 分析 + 重做（重演历史）+ 撤销（写 CLR）。
- 课后思考：为什么 ARIES 的 redo 阶段要连未提交事务的更新也重做?